

Manual de Instalación y Ejecución

Automatización y DevOps con Docker



Introducción

Este manual describe, paso a paso, el proceso de instalación y configuración inicial necesario para automatizar la construcción y publicación de una aplicación contenerizada mediante Git, GitHub y Docker. Cada paso incluye la evidencia visual (captura de pantalla) tomada directamente de la ejecución real en PowerShell.

El objetivo de este documento es proporcionar una guía clara y ordenada que permita replicar el procedimiento de manera sencilla, asegurando que todas las herramientas y servicios involucrados funcionen correctamente entre sí. A través de la integración de Git como sistema de control de versiones, GitHub como plataforma de alojamiento de repositorios y Docker para la creación y gestión de contenedores, se establece una base sólida para implementar procesos de desarrollo modernos y eficientes. Asimismo, este manual sirve como referencia para futuras implementaciones o tareas de mantenimiento, facilitando la comprensión de cada etapa del proceso. La documentación detallada y el respaldo visual de las acciones realizadas permiten verificar los resultados obtenidos, minimizar errores de configuración y promover la estandarización de las prácticas utilizadas durante la automatización de la construcción y publicación de aplicaciones.

Requisitos previos

- Sistema operativo Windows 10 u 11
- Conexión a internet
- PowerShell (incluido por defecto en Windows)
- Cuenta de GitHub
- Docker Desktop instalado y en ejecución

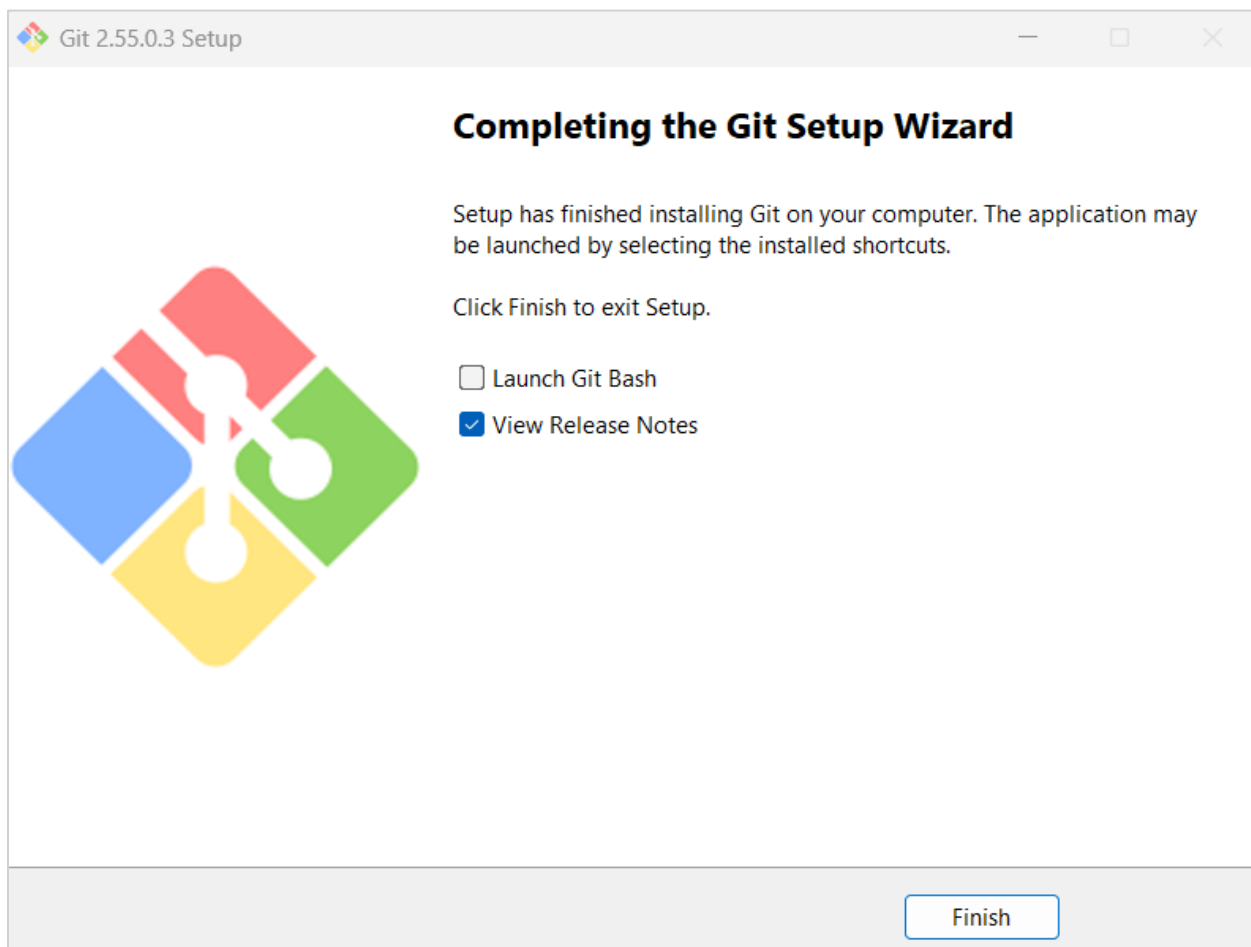
Desarrollo del Manual

Paso 1. Descargar Git para Windows

Se descarga el instalador oficial de Git desde el sitio git-scm.com, herramienta necesaria para el control de versiones y la conexión con GitHub.

Paso 2. Verificar la instalación

Una vez instalado, se confirma que Git quedó correctamente configurado en el sistema.



```
git --version
```

```
Administrador: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> git --version
git version 2.55.0.windows.3
PS C:\WINDOWS\system32>
```

Paso 3. Crear la carpeta del proyecto

Se crea la carpeta que contendrá el proyecto y se ingresa a ella para trabajar dentro.

```
mkdir app
cd app

PS C:\Users\cynth> mkdir app
>> cd app
>> git init

Directorio: C:\Users\cynth

Mode                LastWriteTime         Length Name
----                -
d-----          19/7/2026   20:24         app
Initialized empty Git repository in C:/Users/cynth/app/.git/

PS C:\Users\cynth\app>
```

Paso 4. Configurar la identidad en Git

Git requiere una identidad (nombre y correo) para registrar quién realiza cada cambio. Esta configuración se hace una sola vez por computadora.

```
git config --global user.email "correo@gmail.com"
git config --global user.name "Nombre Apellido"
```

```
ers\cynt\app> git config --global user.email "cynthiavanessa370@gmail.com"

ers\cynt\app> git config --global user.name "Vanessa Rodriguez"
ers\cynt\app>
```

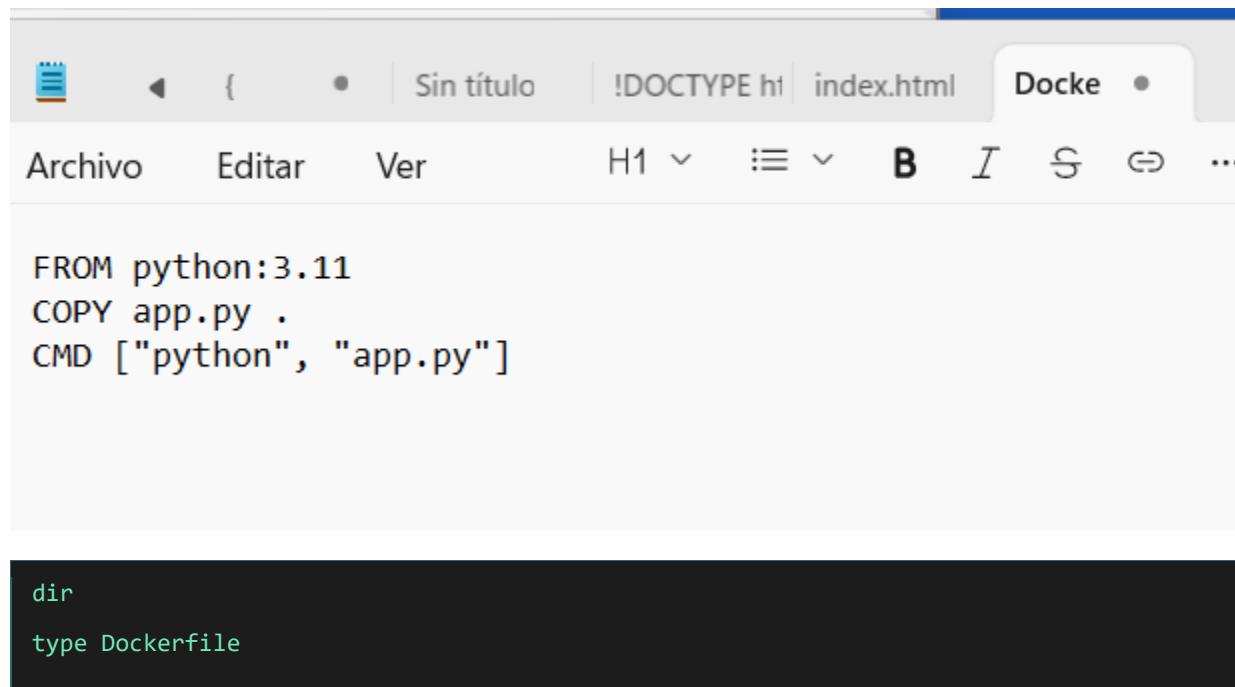
Paso 5. Crear el Dockerfile

El Dockerfile es el archivo que indica a Docker cómo construir la imagen de la aplicación: la base a utilizar, los archivos a copiar y el comando de ejecución.

```
FROM python:3.11
COPY app.py .
CMD ["python", "app.py"]
```

Paso 6. Verificar que los archivos se crearon correctamente

Se revisa el contenido de la carpeta para confirmar que app.py y Dockerfile quedaron creados sin errores ni extensiones incorrectas.



The image shows a code editor window with a tab labeled 'Dockerfile'. The editor displays the following content:

```
FROM python:3.11
COPY app.py .
CMD ["python", "app.py"]
```

Below the editor, a terminal window is open with the following commands and output:

```
dir
type Dockerfile
```

Paso 7. Guardar los cambios en Git (commit local)

Se agregan los archivos al área de preparación de Git y se guarda un punto del historial con un mensaje descriptivo.

```
git add .
git commit -m "primera version"
```

```
PS C:\Users\cynth\app> dir
```

Directorio: C:\Users\cynth\app

Mode	LastWriteTime	Length	Name
-a----	19/7/2026 20:27	58	app.py
-a----	19/7/2026 20:29	57	Dockerfile

```
PS C:\Users\cynth\app>
```

Paso 8. Confirmación del commit exitoso

El resultado confirma que el commit se registró correctamente, indicando los archivos incluidos en este primer punto de guardado del proyecto.

```
PS C:\Users\cynth\app> git add .
>> git commit -m "primera version"
[main (root-commit) c065d21] primera version
2 files changed, 3 insertions(+)
 create mode 100644 Dockerfile
 create mode 100644 app.py
PS C:\Users\cynth\app>
```

Paso 9. Conectar el repositorio local con GitHub

Se enlaza la carpeta local con el repositorio remoto previamente creado en GitHub, indicando la dirección donde se alojará el proyecto.

```
git remote add origin https://github.com/usuario/repositorio.git
```

```
s\cynth\app> git remote add origin https://github.com/Vanessa1356458/app.git
s\cynth\app>
```

Paso 10. Subir el código a GitHub

Se publica el contenido del repositorio local hacia GitHub, dejando el código respaldado en la nube y disponible para el resto del flujo de automatización.

```
git branch -M main
git push -u origin main
```

Paso 11. Conectar tu carpeta local con GitHub y subir el código

El código del proyecto queda publicado correctamente en el repositorio remoto de GitHub, listo para continuar con la configuración de GitHub Actions y Docker Hub.

```
PS C:\Users\cynth\app> git branch -M main
>> git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 369 bytes | 369.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Vanessa1356458/app.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\cynth\app>
```

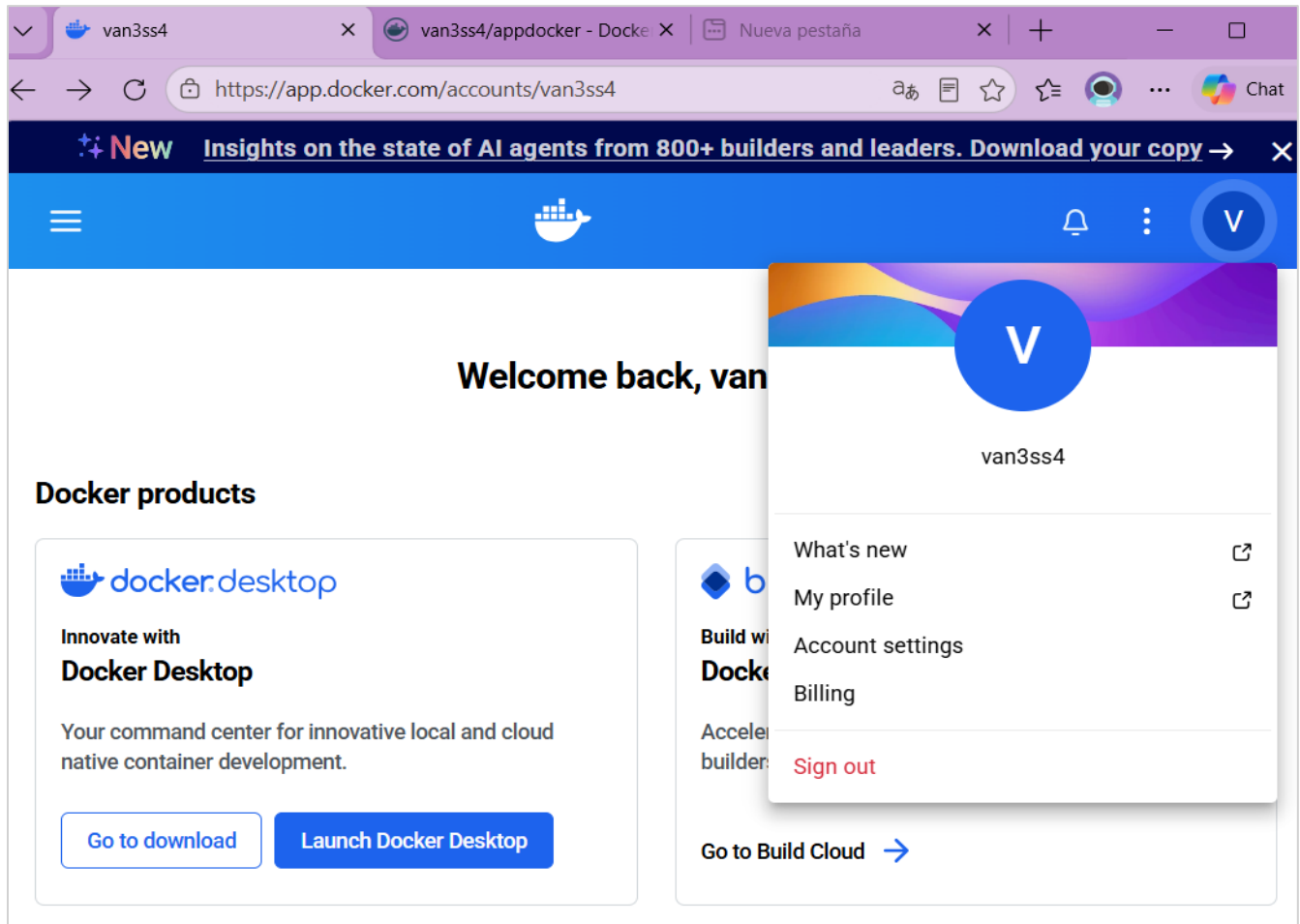
Paso 12. Crear cuenta y repositorio en Docker Hub

Docker Hub es la plataforma donde se publican las imágenes de Docker ya construidas, de forma equivalente a como GitHub aloja el código fuente. Se crea una cuenta y un repositorio público donde posteriormente se publicará la imagen generada automáticamente.

Pasos realizados en el navegador:

- Ingresar a hub.docker.com y crear una cuenta

- Click en "Create Repository"
- Asignar el nombre del repositorio (por ejemplo: appdocker)
- Seleccionar visibilidad "Public"
- Click en "Create"



hub

V

[Repositories](#) / [Create](#)

Using 0 of 1 private repositories. [Upgrade your plan to add more](#)

Create repository

Repository Name *

appdocker



Short description

A short description to identify your repository. If the repository is public, this description is used to index your content on Docker Hub and in search engines, and is visible to users in search results.

Pushing images

You can push a new image to this repository using the CLI:

```
docker tag local-image:tagname
new-repo:tagname
docker push new-repo:tagname
```

Make sure to replace `tagname` with your desired image repository tag.

Visibility

Using 0 of 1 private repositories. [Upgrade your plan to add more](#)

☒ **Public** 

☐ **Private** 

Appears in Docker Hub search results

Only visible to you

Cancel

Create

Explore / van3ss4 / appdocker

van3ss4/appdocker [Manage Repository](#)

By van3ss4 · Updated about 22 hours ago
Grupo #2 Automatización y DevOps con Docker

IMAGE

☆ 0 ↓ 1.5K

Overview **Tags**

Sort by **Newest** Filter tags

TAG
● **latest**
Last pushed about 22 hours by van3ss4

```
docker pull van3ss4/appdocker:latest
```

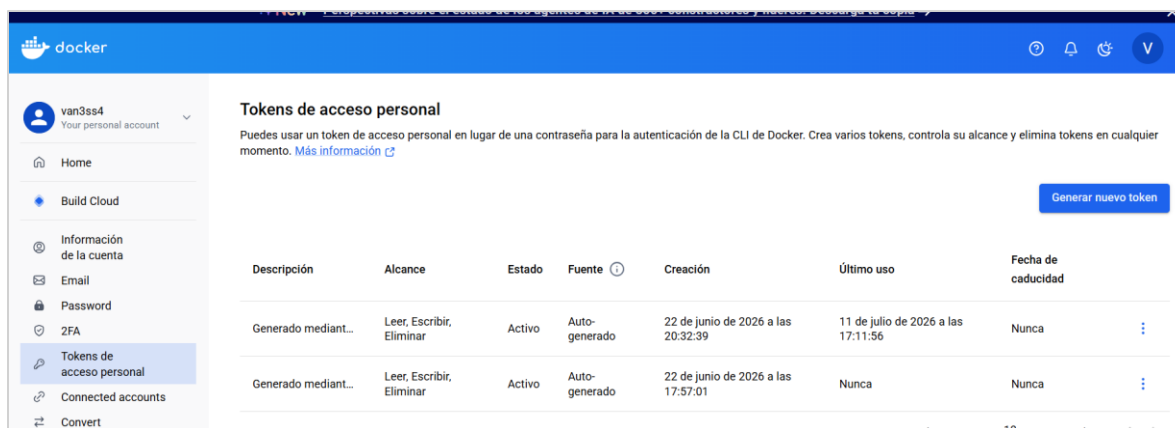
Digest	OS/ARCH	Last pull	Compressed size
67782de5195c	linux/amd64	less than 1 day	390.17 MB

Paso 13. Generar un token de acceso en Docker Hub

Para que GitHub Actions pueda iniciar sesión en Docker Hub de forma automática y seguro, se genera un token de acceso personal, en lugar de utilizar la contraseña real de la cuenta.

Pasos realizados:

- En Docker Hub, ir a Account Settings → Personal access tokens
- Click en "New Access Token"
- Asignar una descripción (por ejemplo: github-actions)
- Seleccionar permisos de "Read & Write"
- Generar y copiar el token de forma inmediata, ya que Docker Hub solo lo muestra una vez



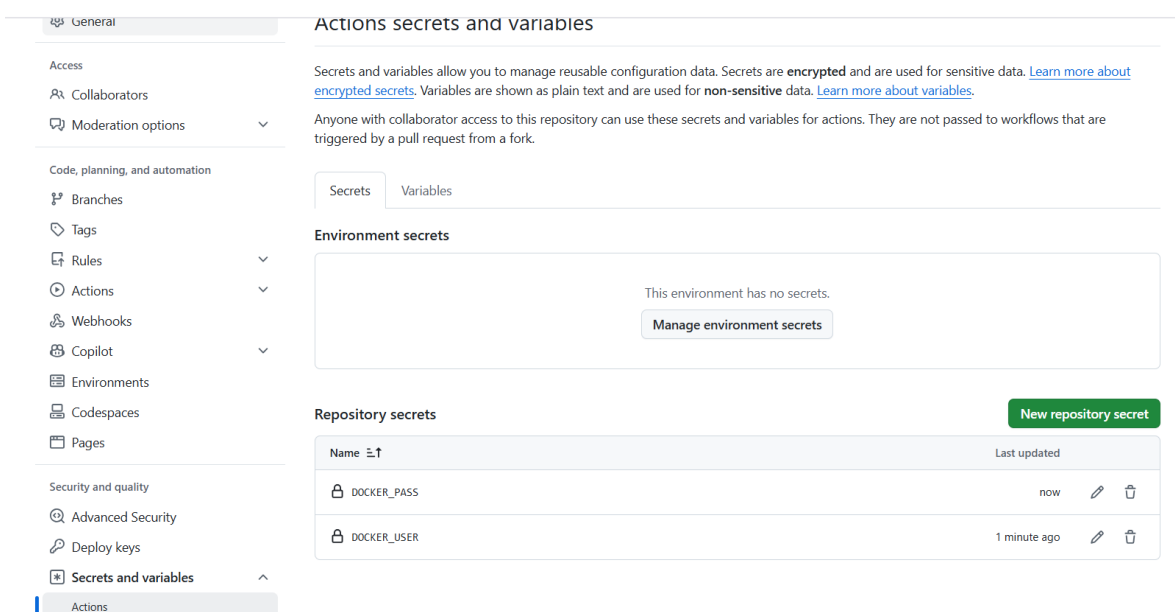
Nota: el token debe tratarse como una contraseña. No debe compartirse ni dejarse escrito en documentos o capturas de pantalla visibles.

Paso 14. Guardar las credenciales de Docker Hub en GitHub (Secrets)

Las credenciales de Docker Hub se almacenan como "Secrets" dentro del repositorio de GitHub, un espacio protegido donde los valores quedan ocultos incluso para quienes tienen acceso al repositorio, y solo son utilizados internamente por GitHub Actions.

Pasos realizados:

- Ingresar al repositorio en GitHub → Settings → Secrets and variables → Actions
- Crear el secret DOCKER_USER, con el valor del usuario de Docker Hub
- Crear el secret DOCKER_PASS, con el valor del token generado en el paso anterior



Paso 15. Configurar GitHub Actions

Se crea el archivo de configuración que define el flujo de automatización. Este archivo debe ubicarse en la ruta exacta `.github/workflows/`, ya que es ahí donde GitHub busca los flujos de trabajo automatizados.

```
mkdir .github
mkdir .github\workflows
notepad .github\workflows\main.yml
Contenido del archivo main.yml:
yaml
name: build-and-push
on:
  push:
    branches: [main]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Login a Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKER_USER }
          password: ${ secrets.DOCKER_PASS }
      - name: Construir y publicar imagen
        uses: docker/build-push-action@v5
        with:
          push: true
          tags: usuario/appdocker:latest
```

Este archivo indica que, cada vez que se suba un cambio a la rama main, GitHub debe iniciar sesión en Docker Hub utilizando los secrets configurados, construir la imagen a partir del Dockerfile, y publicarla automáticamente.

```
PS C:\app> mkdir .github

Directorio: C:\app

Mode                LastWriteTime         Length Name
----                -
d-----          21/7/2026    20:12             .github

PS C:\app> mkdir .github\workflows

Directorio: C:\app\.github

Mode                LastWriteTime         Length Name
----                -
d-----          21/7/2026    20:13             workflows

PS C:\app>
```

```
PS C:\app> New-Item .github\workflows\main.yml
>>

Directorio: C:\app\.github\workflows

Mode                LastWriteTime         Length Name
----                -
-a----          21/7/2026    20:14             0 main.yml

PS C:\app> notepad .github\workflows\main.yml
PS C:\app>
```

```
name: build-and-push

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Login a Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKER_USER }}
          password: ${ secrets.DOCKER_PASS }}

      - name: Construir y publicar imagen
        uses: docker/build-push-action@v5
        with:
          push: true
          tags: van3ss4/appdocker:latest
```

Paso 16. Disparar el flujo automático

Al subir el archivo de configuración al repositorio, se activa por primera vez el flujo de automatización de forma automática.

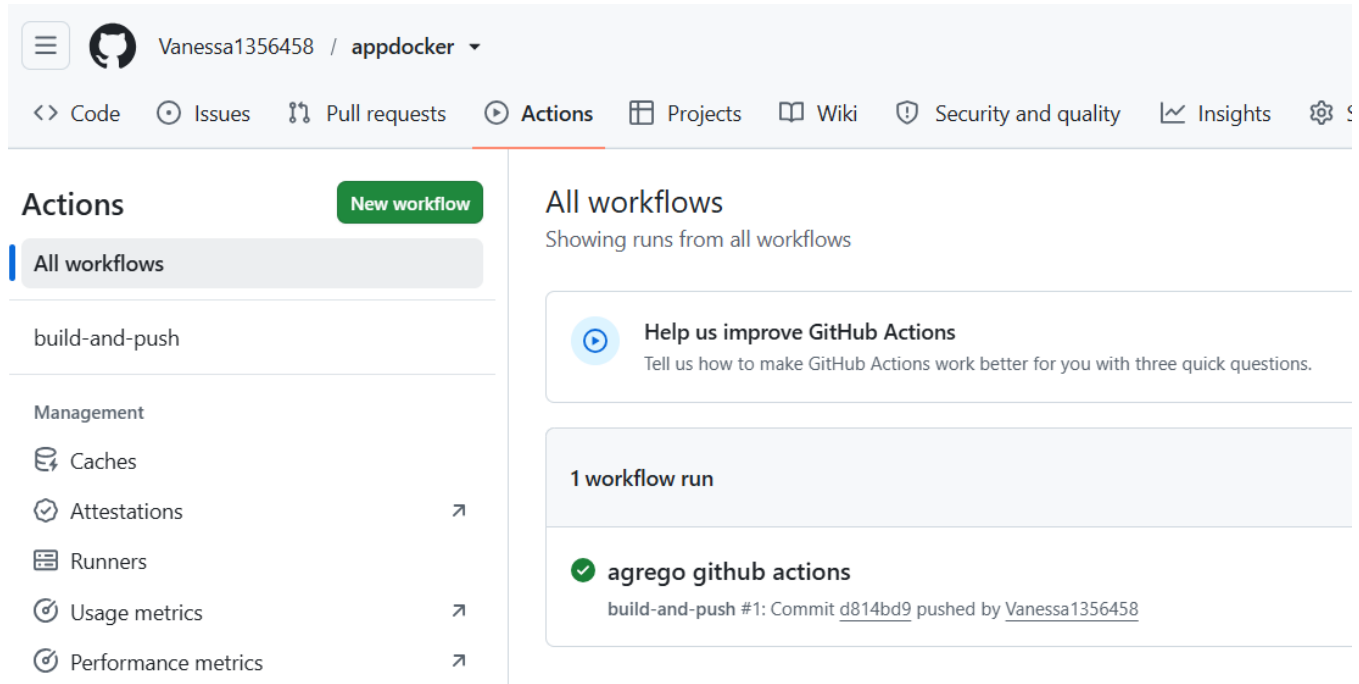
```
git add .
git commit -m "agrego github actions"
```

```
git push
```

```
PS C:\app> git add .
>> git commit -m "agrego github actions"
>> git push
[main d814bd9] agrego github actions
 1 file changed, 23 insertions(+)
 create mode 100644 .github/workflows/main.yml
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 676 bytes | 676.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Vanessa1356458/appdocker.git
 48609a2..d814bd9  main -> main
PS C:\app>
```

Paso 17. Verificar la ejecución del flujo en GitHub Actions

Desde la pestaña "Actions" del repositorio en GitHub, se puede observar el progreso del flujo de trabajo en tiempo real. Una marca verde confirma que el proceso de construcción y publicación de la imagen se completó exitosamente, sin intervención manual.



The screenshot shows the GitHub Actions interface for the repository 'Vanessa1356458 / appdocker'. The 'Actions' tab is active, displaying a list of workflows on the left and a detailed view of the 'build-and-push' workflow on the right. The workflow 'agrego github actions' is shown as completed with a green checkmark. The interface includes a navigation bar with links to Code, Issues, Pull requests, Actions, Projects, Wiki, Security and quality, Insights, and Settings. The 'All workflows' section shows 'Showing runs from all workflows' and a '1 workflow run' section with the completed run 'agrego github actions' by Vanessa1356458.

Paso 18. Descargar la imagen publicada

Para comprobar que la imagen quedó disponible públicamente, se descarga desde Docker Hub como si se hiciera desde un equipo distinto al que la generó.

```
docker pull usuario/appdocker:latest
```

```
PS C:\app> docker pull van3ss4/appdocker:latest
latest: Pulling from van3ss4/appdocker
84c526674046: Pull complete
9e002cae30c4: Pull complete
c33cfc2a5779: Pull complete
65956eab0c1c: Pull complete
b890c9407285: Pull complete
b1cafede9a66: Pull complete
4d58361a56b3: Pull complete
b453535073b6: Pull complete
Digest: sha256:6b4d5876c06aed2ecc122dfd9316b4ecec6f38fc1183ff4632cd742f8dc50c3a1
Status: Downloaded newer image for van3ss4/appdocker:latest
docker.io/van3ss4/appdocker:latest

What's next:
  View a summary of image vulnerabilities and recommendations → docker scout quickview van3ss4/appdock
er:latest
PS C:\app> _
```

Paso 19. Ejecutar la aplicación utilizando la imagen publicada

```
docker run usuario/appdocker:latest
```

Este resultado confirma que la aplicación se ejecuta correctamente a partir de la imagen publicada en Docker Hub, sin requerir el código fuente original ni configuración adicional en el equipo donde se ejecuta.

```
PS C:\app> docker run van3ss4/appdocker:latest
Hola desde Docker
PS C:\app>
```

Conclusión

Este manual documentó el proceso completo de instalación y configuración necesario para automatizar la construcción y publicación de una aplicación contenerizada. A través de Git y GitHub se gestionó el control de versiones del código; mediante GitHub Actions se automatizó la construcción de la imagen; y con Docker Hub se logró su publicación y distribución. El resultado final demuestra un flujo de integración y despliegue continuo (CI/CD) funcional, donde cada cambio en el código se traduce automáticamente en una nueva versión de la aplicación, lista para ejecutarse en cualquier equipo